Currency Converter:

Create a script that converts between different currencies using exchange rates.

```
import requests

def convert_currency(amount, from_currency, to_currency
    url = f"https://api.exchangerate-api.com/v4/latest/
    response = requests.get(url)
    data = response.json()
    exchange_rate = data['rates'][to_currency]
    converted_amount = amount * exchange_rate
    return converted_amount
```

```
amount = float(input("Enter the amount: "))
from_currency = input("From currency: ").upper()
to_currency = input("To currency: ").upper()
```

```
Enter the amount: 450
From currency: PKR
To currency: USD
```

```
result = convert_currency(amount, from_currency, to_currency)
print(f"{amount} {from_currency} is equal to {result} {to_currency}")
```

```
450.0 PKR is equal to 1.611 USD
```

Weather Checker:

Create a script that fetches and displays the current weather for a given location.

```
import requests

def get_weather(city):
    api_key = 'YOUR_API_KEY'
    url = f'http://api.openweathermap.org/data/2.5/weat
    response = requests.get(url)
    data = response.json()
    weather_description = data['weather'][0]['descripti
    temperature = data['main']['temp']
    return weather_description, temperature
    city = input("Enter the city: ")
weather, temperature = get_weather(city)
```

```
weather, temperature = get_weather(city)
print(f"Weather in {city}: {weather}")
print(f"Temperature: {temperature}°C")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_7297/3978943992.py in <cell line: 0>()
     10     return weather_description, temperature
     11     city = input("Enter the city: ")
---> 12 weather, temperature = get_weather(city)
     13 print(f"Weather in {city}: {weather}")
     14 print(f"Temperature: {temperature}°C")

NameError: name 'city' is not defined
```

Port Scanning

```
import socket
import threading

# Define the target and port range to scan
target = 'portals.au.edu.pk'
port_range = (1, 1024)

# Function to scan a range of ports on the target
def scan_port(target, port):
    try:
        # Create a socket object
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Set a timeout for the connection attempt
        socket.setdefaulttimeout(1)

        # Attempt to connect to the target's IP address and port
        result = s.connect_ex((target, port))

        if result == 0:
            print(f"Port {port} is open")

        s.close()

    except KeyboardInterrupt:
        print("Scanning stopped by user.")
        exit()

    except socket.gaierror:
```

```
            print("Hostname could not be resolved.")
            exit()

        except socket.error:
            print("Could not connect to the server.")
            exit()

    # Multi-threaded port scanning
    def scan_ports(target, port_range):
        print(f"Scanning target {target}...")
        for port in range(port_range[0], port_range[1] + 1):
            thread = threading.Thread(target=scan_port, args=(target, port))
            thread.start()

    # Main function
    def main():
        try:
            scan_ports(target, port_range)
        except KeyboardInterrupt:
            print("Scanning stopped by user.")
            exit()

    if __name__ == "__main__":
        main()
```

```
    import socket
    import threading
    import concurrent.futures

    # Function to scan a single port on the target
    def scan_port(target, port):
        try:
            # Create a socket object
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
                # Set a timeout for the connection attempt
                s.settimeout(1)
                # Attempt to connect to the target's IP address and port
                result = s.connect_ex((target, port))
                if result == 0:
                    # Banner grabbing to retrieve service information
                    banner = s.recv(1024).decode().strip()
                    print(f"Port {port} is open. Service: {banner}")
        except KeyboardInterrupt:
            print("Scanning stopped by user.")
            exit()
        except socket.gaierror:
            print("Hostname could not be resolved.")
            exit()
        except socket.error:
```

```
            pass

    # Multi-threaded port scanning
    def scan_ports(target, port_range, num_threads):
        print(f"Scanning target {target}...")
        with concurrent.futures.ThreadPoolExecutor(max_workers=num_threads) as exec
            for port in range(port_range[0], port_range[1] + 1):
                executor.submit(scan_port, target, port)

    # Main function
    def main():
        try:
            target = input("Enter target IP address or domain: ")
            start_port = int(input("Enter start port: "))
            end_port = int(input("Enter end port: "))
            num_threads = int(input("Enter number of threads (e.g., 10): "))

            if start_port > end_port:
                print("Start port cannot be greater than end port.")
                return

            port_range = (start_port, end_port)

            scan_ports(target, port_range, num_threads)

        except KeyboardInterrupt:
            print("Scanning stopped by user.")
            exit()
        except ValueError:
            print("Invalid input. Please enter a valid number.")
            exit()

    if __name__ == "__main__":
        main()
```

Python Libraries

Requests:

Description: A library for making HTTP requests. It's useful for interacting with web services and APIs. Example: Create a script to fetch weather data from a weather API and display it.

```
    import requests
```

```
# OpenWeatherMap API endpoint and API key (replace with your own API key)
api_endpoint = "https://api.openweathermap.org/data/2.5/weather"
api_key = "YOUR_API_KEY"

# Function to fetch weather data for a city
def get_weather(city):
    params = {
        'q': city,
        'appid': api_key,
        'units': 'metric'  # Units for temperature (e.g., Celsius)
    }

    try:
        response = requests.get(api_endpoint, params=params)
        data = response.json()

        if response.status_code == 200:
            # Extract weather information from the API response
            temperature = data['main']['temp']
            description = data['weather'][0]['description']
            humidity = data['main']['humidity']
            wind_speed = data['wind']['speed']

            # Display weather information
            print(f"Weather in {city}:")
            print(f"Temperature: {temperature}°C")
            print(f"Description: {description}")
            print(f"Humidity: {humidity}%")
            print(f"Wind Speed: {wind_speed} m/s")
        else:
            print("Failed to fetch weather data. Please check the city name anc

    except requests.exceptions.RequestException as e:
        print(f"Error: {e}")

# Main function
def main():
    city = input("Enter the name of the city: ")
    get_weather(city)

if __name__ == "__main__":
    main()
```

NumPy:

Description: A library for numerical computing in Python. It provides support for large, multi-dimensional arrays and matrices. Example: Create a script to perform matrix

multiplication for solving linear equations.

```
import numpy as np

# Define the coefficients matrix A and the constants vector B
A = np.array([[2, 3], [1, -2]])
B = np.array([8, 1])

# Solve the system of linear equations using matrix multiplication
X = np.linalg.solve(A, B)

# Display the solution
print("Solution to the system of linear equations:")
print("x =", X[0])
print("y =", X[1])
```

Pandas:

Description: A library for data manipulation and analysis. It's used for data cleaning, exploration, and transformation. Example: Create a script to analyze a dataset, calculate statistics, and generate visualizations.

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample dataset (you can replace this with your own dataset)
data = {
    'Date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023-01-0
    'Product': ['A', 'B', 'A', 'C', 'B'],
    'Sales': [100, 150, 120, 80, 200]
}

# Create a Pandas DataFrame from the dataset
df = pd.DataFrame(data)

# Display the first few rows of the DataFrame
print("Sample Data:")
print(df)

# Calculate basic statistics
print("\nStatistics:")
print(df.describe())

# Group data by 'Product' and calculate total sales per product
product_sales = df.groupby('Product')['Sales'].sum().reset_index()
```

```
# Generate a bar chart of total sales per product
plt.figure(figsize=(8, 6))
plt.bar(product_sales['Product'], product_sales['Sales'])
plt.title("Total Sales per Product")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.show()
```

Matplotlib:

Description: A library for creating static, animated, and interactive visualizations in Python. Example: Create a script to plot a line chart showing stock price trends over time.

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import numpy as np

# Generate fictional stock price data (replace with real data)
dates = np.array(['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04', '2023
prices = np.array([100, 105, 110, 115, 112])

# Convert date strings to datetime objects for better plotting
dates = [mdates.datestr2num(date) for date in dates]

# Create a figure and axis
fig, ax = plt.subplots()

# Plot the stock price data as a line chart
ax.plot(dates, prices, marker='o', linestyle='-')

# Customize the plot
ax.set(xlabel='Date', ylabel='Stock Price',
       title='Stock Price Trends Over Time')
ax.grid()

# Format the x-axis to display dates nicely
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y-%m-%d"))

# Rotate x-axis labels for better readability
plt.xticks(rotation=45)

# Show the plot
plt.tight_layout()
plt.show()
```

Scikit-learn:

Description: A machine learning library for Python. It provides tools for classification, regression, clustering, and more. Example: Create a script to build and train a machine learning model for sentiment analysis.

```
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report

# Sample movie review dataset (you can replace this with your own dataset)
reviews = [
```

```
    "This movie is fantastic!",
    "I loved it.",
    "Not bad, but not great either.",
    "The worst movie I've ever seen.",
    "Terrible acting and a boring plot."
]

labels = [1, 1, 0, 0, 0]  # 1 for positive sentiment, 0 for negative sentiment

# Convert text data into numerical features using CountVectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(reviews)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.2, r

# Create and train a Multinomial Naive Bayes classifier
classifier = MultinomialNB()
classifier.fit(X_train, y_train)

# Make predictions on the test set
y_pred = classifier.predict(X_test)

# Evaluate the model's accuracy and performance
accuracy = accuracy_score(y_test, y_pred)
classification_rep = classification_report(y_test, y_pred)

# Display the results
print("Accuracy:", accuracy)
print("\nClassification Report:\n", classification_rep)
```

OpenCV:

Description: A computer vision library that provides tools for image and video analysis, processing, and manipulation. Example: Create a script to detect and track objects in a live video stream.

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Initialize video capture from a sample video URL
video_url = "https://www.sample-videos.com/video123/mp4/720/big_buck_bunny_720p
cap = cv2.VideoCapture(video_url)

# Define the lower and upper bounds of the red color in HSV format
lower_red = np.array([0, 100, 100])
```

```
    upper_red = np.array([10, 255, 255])

    # Create a kernel for morphological operations (used for noise reduction)
    kernel = np.ones((5, 5), np.uint8)

    while True:
        # Read a frame from the video
        ret, frame = cap.read()

        if not ret:
            break

        # Convert the frame to the HSV color space
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

        # Create a mask for the red color
        mask = cv2.inRange(hsv, lower_red, upper_red)

        # Apply morphological operations to the mask (noise reduction)
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)

        # Find contours in the mask
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SI

        # Draw bounding boxes around detected objects
        for contour in contours:
            area = cv2.contourArea(contour)
            if area > 100:  # Adjust the area threshold as needed
                x, y, w, h = cv2.boundingRect(contour)
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

        # Display the processed frame
        cv2_imshow(frame)

        # Break the loop when 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    # Release the video capture
    cap.release()
```

Beautiful Soup:

Description: A library for web scraping purposes. It helps parse HTML and XML documents and extract information. Example: Create a script to scrape product information from an e-commerce website.

```
import requests
from bs4 import BeautifulSoup

# Define the URL of the e-commerce website to scrape
url = 'https://example.com/products'

# Send an HTTP GET request to the URL
response = requests.get(url)

# Check if the request was successful (status code 200)
if response.status_code == 200:
    # Parse the HTML content of the page using Beautiful Soup
    soup = BeautifulSoup(response.text, 'lxml')

    # Find and extract product information
    products = []
    product_elements = soup.find_all('div', class_='product')

    for product_element in product_elements:
        # Extract product name
        product_name = product_element.find('h2', class_='product-name').text.s

        # Extract product price
        product_price = product_element.find('span', class_='product-price').te

        # Add product information to the list
        products.append({
            'name': product_name,
            'price': product_price
        })

    # Print the scraped product information
    for product in products:
        print(f"Product Name: {product['name']}")
        print(f"Product Price: {product['price']}")
else:
    print("Failed to retrieve the web page.")
```

Django:

Description: A high-level Python web framework for building web applications. It follows the "batteries-included" philosophy. Example: Create a web-based calculator application using Django to perform arithmetic operations.

Flask:

Description: A lightweight and micro web framework for building web applications and RESTful APIs. Example: Create a RESTful API for a calculator that can add, subtract, multiply, and divide numbers.

SymPy:

Description: A Python library for symbolic mathematics. It's used for algebraic operations and symbolic math. Example: Create a script to solve algebraic equations and find roots symbolically.

```
import sympy as sp

# Define symbolic variables
x, y = sp.symbols('x y')

# Define an algebraic equation
equation = sp.Eq(x**2 + 3*x - 4, 0)

# Solve the equation symbolically
solutions = sp.solve(equation, x)

# Display the solutions
print("Solutions to the equation:")
for sol in solutions:
    print(f"x = {sol}")

# Find the roots of a polynomial
polynomial = x**3 - 6*x**2 + 11*x - 6
roots = sp.roots(polynomial)

# Display the roots
print("\nRoots of the polynomial:")
for root, multiplicity in roots.items():
    print(f"Root: {root}, Multiplicity: {multiplicity}")
```

Pillow:

Description: A library for working with images in Python. It allows you to open, manipulate, and save various image formats. Example: Create a script to resize and apply filters to images.

```
from PIL import Image, ImageFilter
import requests
from io import BytesIO

# Function to download an image from a URL
def download_image(url):
    response = requests.get(url)
    return Image.open(BytesIO(response.content))

# Function to resize an image
def resize_image(image, width, height):
    return image.resize((width, height))

# Function to apply a filter to an image
def apply_filter(image, filter_type):
    if filter_type == 'blur':
        return image.filter(ImageFilter.BLUR)
    elif filter_type == 'sharpen':
        return image.filter(ImageFilter.SHARPEN)
    elif filter_type == 'edge':
        return image.filter(ImageFilter.FIND_EDGES)
    else:
        return image

# Main function
def main():
    # Replace this URL with the URL of your choice
    image_url = "https://images.nationalgeographic.org/image/upload/t_edhub_res

    image = download_image(image_url)

    print("1. Resize Image")
    print("2. Apply Blur Filter")
    print("3. Apply Sharpen Filter")
    print("4. Apply Edge Detection Filter")
    choice = input("Enter your choice (1/2/3/4): ")

    if choice == '1':
        width = int(input("Enter the new width: "))
        height = int(input("Enter the new height: "))
        resized_image = resize_image(image, width, height)
        resized_image.show()
        resized_image.save("resized_image.jpg")
        print("Resized image saved as 'resized_image.jpg'")
    elif choice in ['2', '3', '4']:
        filter_type = {
            '2': 'blur',
            '3': 'sharpen',
            '4': 'edge'
        }[choice]
```

```
            filtered_image = apply_filter(image, filter_type)
            filtered_image.show()
            filtered_image.save(f"{filter_type}_image.jpg")
            print(f"{filter_type.capitalize()} filter applied. Image saved as '{fil
        else:
            print("Invalid choice. Exiting.")

    if __name__ == "__main__":
        main()
```

Tkinter:

Description: The standard GUI library for Python. It allows you to create desktop GUI applications. Example: Build a graphical calculator application with buttons and an input field.

```
import tkinter as tk

def button_click(number):
    current = entry.get()
    entry.delete(0, tk.END)
    entry.insert(0, current + str(number))

def clear():
    entry.delete(0, tk.END)

def calculate():
    try:
        result = eval(entry.get())
        entry.delete(0, tk.END)
        entry.insert(0, str(result))
    except Exception:
        entry.delete(0, tk.END)
        entry.insert(0, "Error")

# Create the main window
root = tk.Tk()
root.title("Calculator")

# Create an entry widget for input and display
entry = tk.Entry(root, width=20, font=("Helvetica", 20))
entry.grid(row=0, column=0, columnspan=4)

# Define the buttons
buttons = [
    '7', '8', '9', '/',
    '4', '5', '6', '*',
```

```
        '1', '2', '3', '-',
        '0', 'C', '=', '+'
    ]

    # Create and place the buttons on the grid
    row_val = 1
    col_val = 0

    for button in buttons:
        if button == '=':
            tk.Button(root, text=button, padx=20, pady=20, font=("Helvetica", 20),
        elif button == 'C':
            tk.Button(root, text=button, padx=20, pady=20, font=("Helvetica", 20),
        else:
            tk.Button(root, text=button, padx=20, pady=20, font=("Helvetica", 20),

        col_val += 1
        if col_val > 3:
            col_val = 0
            row_val += 1

    # Run the Tkinter main loop
    root.mainloop()
```

NLTK (Natural Language Toolkit):

Description: A library for working with human language data. It provides tools for text analysis and natural language processing. Example: Create a script to perform sentiment analysis on a collection of text reviews.

```
import nltk
nltk.download('vader_lexicon')
```

```
import nltk
from nltk.sentiment.vader import SentimentIntensityAnalyzer

# Initialize the VADER sentiment analyzer
sid = SentimentIntensityAnalyzer()

# Sample collection of text reviews
reviews = [
    "This product is amazing! I love it.",
    "The service was terrible, and I'm really disappointed.",
    "The quality of the book is excellent.",
    "I don't like this at all.",
```

```
]

# Perform sentiment analysis on each review
for review in reviews:
    sentiment_scores = sid.polarity_scores(review)

    # Determine sentiment based on the compound score
    if sentiment_scores['compound'] >= 0.05:
        sentiment = "Positive"
    elif sentiment_scores['compound'] <= -0.05:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"

    print(f"Review: {review}")
    print(f"Sentiment: {sentiment}")
    print(f"Sentiment Scores: {sentiment_scores}")
    print()
```

Pygame:

Description: A cross-platform set of Python modules designed for writing video games. It's also used for multimedia applications. Example: Create a simple 2D game, like a platformer or a puzzle game.

```
import pygame
import random

# Initialize Pygame
pygame.init()

# Set up game constants
WIDTH, HEIGHT = 800, 600
PLAYER_SIZE = 50
PLAYER_SPEED = 5
WHITE = (255, 255, 255)

# Create the game window
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Simple Pygame Game")

# Define the player character
player = pygame.Surface((PLAYER_SIZE, PLAYER_SIZE))
player.fill((0, 128, 255))
player_rect = player.get_rect()
player_rect.center = (WIDTH // 2, HEIGHT // 2)
```

```
# Create an enemy character
enemy = pygame.Surface((PLAYER_SIZE, PLAYER_SIZE))
enemy.fill((255, 0, 0))
enemy_rect = enemy.get_rect()
enemy_rect.center = (random.randint(0, WIDTH), random.randint(0, HEIGHT))

# Set up a clock to control the game's frame rate
clock = pygame.time.Clock()

# Main game loop
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # Handle player input
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player_rect.x -= PLAYER_SPEED
    if keys[pygame.K_RIGHT]:
        player_rect.x += PLAYER_SPEED
    if keys[pygame.K_UP]:
        player_rect.y -= PLAYER_SPEED
    if keys[pygame.K_DOWN]:
        player_rect.y += PLAYER_SPEED
```

```
# Check for collisions
    if player_rect.colliderect(enemy_rect):
        enemy_rect.center = (random.randint(0, WIDTH), random.randint(0, HEIGHT
```

```
# Fill the screen with white
    screen.fill(WHITE)
```

```
# Draw the player and enemy characters
    screen.blit(player, player_rect)
    screen.blit(enemy, enemy_rect)
```

```
# Update the display
    pygame.display.update()
```

```
 # Cap the frame rate
    clock.tick(60)
```

```
# Quit Pygame
pygame.quit()
```

NetworkX:

Description: A library for studying the structure and dynamics of complex networks (graphs). Example: Create a script to analyze the social network connections of users on a platform.

```
import networkx as nx
import matplotlib.pyplot as plt

# Create an empty graph
social_network = nx.Graph()

# Add nodes (representing users)
social_network.add_node("Alice")
social_network.add_node("Bob")
social_network.add_node("Charlie")
social_network.add_node("David")

# Add edges (representing connections between users)
```